4주차: 트리 재귀·BFS·BST

계층 기록의 깊이와 검색 순서

하위 작업이 최대 두 개씩 연결된 작업 트리에서 가장 깊은 단계는 어디일까요? 단계별 현황을 출력하고, 검색용 트리가 정렬 규칙을 지키는지도 확인합니다.

한 줄로 저장한 기록과 달리 각 노드에서 두 갈래를 따라가야 합니다. 재귀 반환값, 스택·큐, 조상의 범위로 필요한 정보를 전달합니다.

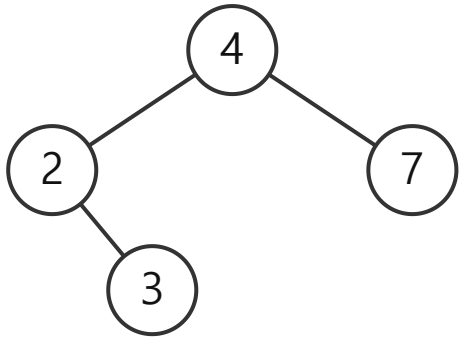
4주차 문제와 학습 목표

7차시: 재귀 반환값과 최대 깊이. 8차시: BFS·BST 검증·중위 순회.

- 빈 트리와 자식 호출의 반환값 정의
- 한 레벨의 큐 경계와 조상 범위 추적
- 시간·보조 공간·출력 공간 구분
- 작은 예제로 재귀를 이해하고 반복 구현으로 긴 트리 처리

실습: LeetCode 104, 102, 98, 230. 그림·추적 표·중간 점검은 강의용 추가 예시입니다.

이진 트리의 노드와 입력 표기



`[4,2,7,None,3]`: 레벨 순서 표기. `None`은 없는 자식이며, 다음 값은 큐에서 꺼낸 부모의 왼쪽·오른쪽 자식에 배정합니다.

깊이: 루트부터 세는 노드 수. 빈 트리 0, 루트만 있으면 1.

TreeNode와 지역 변수

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

- `left`, `right`: 자식 노드 참조
- 재귀 호출마다 별도의 지역 변수
- `return`: 계산 결과를 호출자에게 전달

각 자식의 높이를 반환받아 현재 높이를 계산합니다.

104 · 문제와 상태

LeetCode 104: Maximum Depth of Binary Tree · Easy

root에서 잎까지 가장 긴 경로의 노드 수를 반환합니다.

```
root=[4,2,7,None,3] → 3
```

작은 트리에서 depth(None)=0, depth(node)=1+max(left,right)를 도출합니다. 실행 해설은 (node,depth)의 명시적 스택도 제공합니다.

최대 깊이의 재귀식

```
def depth_recursive(root):
    """Small-tree demonstration; Python's recursion limit applies."""
    if root is None:
        return 0
    return 1 + max(depth_recursive(root.left), depth_recursive(root.right))
```

빈 트리의 높이는 0. 자식 높이 중 큰 값에 현재 노드 1을 더합니다. 이 함수는 작은 트리의 설명용입니다.

부분 트리의 반환값

호출	왼쪽 높이	오른쪽 높이	반환
depth(3)	0	0	1
depth(2)	0	1	2
depth(7)	0	0	1
depth(4)	2	1	3

부모는 두 자식의 결과를 받은 뒤 현재 높이를 계산합니다.

최대 깊이의 명시적 스택

```
def max_depth(root):
    """Explicit stack supports skew trees within the official size limit."""
    if root is None:
        return 0
    stack, best = [(root, 1)], 0
    while stack:
        node, depth = stack.pop()
        best = max(best, depth)
        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))
    return best
```

104 · 정당성과 비용

재귀는 자식 높이가 맞다고 가정하면 현재 높이를 정확히 결합합니다. 명시적 스택은 각 노드의 실제 root 깊이를 전달하며 최대를 기록하므로 같은 답입니다.

n=노드 수,h=높이. O(n) 시간, DFS 보조 O(h), 최악 O(n).

분기 누락과 재귀 깊이

`if left ... elif right ...`는 왼쪽 자식이 있으면 오른쪽을 건너뜁니다. 양쪽 자식은 별도의 `if`로 검사합니다.

Python의 재귀 제한보다 긴 연결 모양 트리에서는 재귀 호출이 실패할 수 있습니다. 공식 최대 크기 10,000노드에는 명시적 스택 구현을 사용합니다.

검사: 빈 트리, 노드 하나, 오른쪽만 긴 트리.

102 · 문제와 상태

LeetCode 102: Binary Tree Level Order Traversal · Medium

트리 값을 깊이별 왼쪽부터 묶어 반환합니다.

`root=[4,2,7,None,3] → [[4],[2,7],[3]]`

deque에서 시작 시점의 len(queue)개만 꺼내고 자식을 다음 레벨로 추가합니다.

deque와 레벨 경계

```
from collections import deque
queue = deque([root])
```

- `popleft()`: 가장 먼저 들어온 노드 제거
- `append(child)`: 다음 레벨의 자식 추가
- `range(len(queue))`: 레벨 시작 시점의 노드 수 고정

처리 중 추가한 자식은 다음 외부 반복에서 방문합니다.

레벨별 큐 추적

시작 큐	이번에 꺼낼 수	결과 레벨	다음 큐
[4]	1	[4]	[2,7]
[2,7]	2	[2,7]	[3]
[3]	1	[3]	[]

현재 레벨의 노드를 모두 꺼내고 나면 큐에는 다음 레벨만 남습니다. 자식은 왼쪽부터 추가합니다.

BFS 레벨 순회 구현

```
def level_order(root):
    if root is None:
        return []
    queue, result = deque([root]), []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

deque는 collections에서 가져옵니다.

102 · 정당성과 비용

각 외부 반복 시작 때 큐에는 현재 레벨만 있습니다. 그 수만 꺼내면 새 자식은 다음 레벨 큐로 남습니다. 왼쪽 자식부터 추가하면 순서도 유지됩니다.

n=노드 수,w=최대 폭. O(n) 시간, 큐 O(w), 출력 O(n).

큐 연산과 결과 저장

`list.pop(0)`는 남은 원소를 이동시킵니다. 넓은 트리에서 반복하면 전체 비용이 $O(n^2)$까지 증가합니다. `deque.popleft()`의 제거 비용은 $O(1)$입니다.

큐에는 현재 레벨의 남은 노드와 다음 레벨의 일부가 함께 있을 수 있습니다. 두 레벨의 폭은 각각 최대 w이므로 큐 공간은 $O(w)$입니다.

반환하는 레벨 목록은 별도로 $O(n)$ 공간을 사용합니다.

98 · 문제와 상태

LeetCode 98: Validate Binary Search Tree · Medium

모든 왼쪽 후손이 작고 오른쪽 후손이 큰 BST인지 판정합니다.

`root=[8,3,10,None,None,6,12] → False`

각 노드에 열린 범위 (lower,upper)를 전달합니다. 왼쪽 upper=node.val, 오른쪽 lower=node.val입니다.

조상에서 전달한 열린 범위

- 루트: `(-inf, inf)`
- 왼쪽 자식: `(lower, node.val)`
- 오른쪽 자식: `(node.val, upper)`
- 검사: `lower < node.val < upper`

엄격한 부등식이므로 중복 값은 허용하지 않습니다. 예를 들어 루트 8의 오른쪽에 있는 모든 후손은 8보다 커야 합니다.

조상 범위 위반의 반례

노드	lower	upper	판정
8	-inf	inf	통과
10	8	inf	통과
6	8	10	6<=8, 실패

루트에서 6으로 가는 경로만 표시했습니다. 6은 부모 10과의 비교는 통과하지만 조상에게서 받은 하한 8을 위반합니다.

BST 검증 구현

```
def valid_bst(root):
    stack = [(root, float("-inf"), float("inf"))]
    while stack:
        node, lower, upper = stack.pop()
        if node is None:
            continue
        if not lower < node.val < upper:
            return False
        stack.append((node.right, node.val, upper))
        stack.append((node.left, lower, node.val))
    return True
```

98 · 정당성과 비용

범위는 모든 조상에서 온 제약의 교집합입니다. 범위를 만족하면 조상 관계를 모두 만족하고, 자식 범위를 줄여 같은 성질을 유지합니다. 한 위반이라도 전체가 False입니다.

n=노드 수,h=높이. O(n) 시간, 명시적 DFS 스택 O(h), 최악 O(n).

230 · 문제와 상태

LeetCode 230: Kth Smallest Element in a BST · Medium

BST에서 작은 순서로 k번째 값을 반환합니다.

`root=[5,2,8,1,3], k=3 → 3`

명시적 스택으로 inorder 순회하고 노드를 꺼낼 때 k를 하나 줄입니다.

중위 순회와 방문 순서

BST [5,2,8,1,3]의 중위 순서: 1,2,3,5,8.

방문 값	방문 후 남은 k	k=3일 때 행동
1	2	계속
2	1	계속
3	0	3 반환

왼쪽으로 내려가며 노드를 저장하고, 꺼낸 노드를 방문한 뒤 오른쪽 부분 트리로 이동합니다.

k번째 작은 값 구현

```
def kth_smallest(root, k):
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right
    raise ValueError("k exceeds tree size")
```

230 · 정당성과 비용

BST의 왼쪽 전체는 현재보다 작고 오른쪽 전체는 큽니다. 귀납적으로 inorder는 정렬 순서이므로 k번째 방문 값이 답입니다.

n=노드 수,h=높이. O(h+k) 시간(최악 O(n)), O(h) 보조 공간.

기준 풀이와 비용 비교

문제	기준 풀이	이번 구현
최대 깊이	작은 트리 재귀	명시적 DFS 스택
레벨 순회	리스트 앞에서 제거	deque의 O(1) 제거
BST 검증	각 노드에서 후손 재검사: 최악 $O(n^2)$	범위 전달: O(n)
k번째 값	중위 배열 전체 저장: O(n) 시간·공간	조기 종료: O(h+k) 시간, O(h) 공간

높이 h는 편향 트리에서 n까지 증가합니다. 정수 비교·노드 참조는 상수 비용으로 가정합니다.

중간 점검과 제출물

1. `[4,2,7,None,3]`의 최대 깊이와 레벨별 값을 실행 전에 적습니다.
2. `[8,3,10,None,None,6,12]`가 BST인지 근거와 함께 판정합니다.
3. BST `[5,2,8,1,3]`에서 k=1, 3, 5의 반환값을 구합니다.

코드·상태 정의·정당성·비용·경계 입력 제출. 학생 실습을 먼저 풀고 해설과 비교합니다.

실행과 참고자료

4주차 코드를 풀고 `code` 폴더에서 실행합니다.

```
python demo_07.py
python demo_08.py
python verify.py
python verify.py --student
```

마지막 명령은 `student_week_04.py`를 구현한 뒤 실행합니다.

Erik Demaine·Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).